```
/*
================================================================================
JAVASCRIPT COMPLETE INTERVIEW + REVISION GUIDE
Pasteable Single-File Version
================================================================================

Author: M365 Copilot
Purpose:
- Designed to be pasted into a .js file for learning and revision
- Highly commented for interview preparation
- Includes theory + why + use case + limitations + examples + outputs
- Browser-only and Node-only items are clearly labeled

How to study:
1) Read comments first
2) Predict outputs before uncommenting console.log lines
3) Run examples section by section
4) Re-explain each topic in your own words

Interview Answer Formula:
1) What is it?
2) Why does it exist?
3) When is it used?
4) Example
5) Limitations
6) Best practice / advanced note
*/

//
================================================================================
// [00] QUICK INTERVIEW STRATEGY
//
================================================================================
/*
When answering in interviews, try this sequence:

1) Definition
   - What exactly is the concept?

2) Purpose
   - Why was it introduced / why does it exist?

3) Use case
   - Where is it used in real projects?

4) Example
```

- Show a small example

5) Limitation
- Mention trap / drawback / edge case

6) Best practice
- Explain how to avoid problems
*/

//
==========
// [01] JAVASCRIPT BASICS
//
==========
/*
What is JavaScript?
- JavaScript is a high-level, dynamically typed, prototype-based,
  multi-paradigm programming language.
- Initially it became popular for adding behavior to web pages.
- Today it is used in browsers, servers, mobile apps, desktop apps, tooling,
  automation, and edge/serverless platforms.

Breakdown of the definition:

1) High-level
- Easier for humans to read/write than low-level languages like C/Assembly
- You do not manually manage memory

2) Dynamically typed
- Variable types are determined at runtime

3) Prototype-based
- Objects inherit through prototypes, not only classical class systems

4) Multi-paradigm
- Supports procedural, object-oriented, and functional programming styles
*/

// JavaScript vs ECMAScript
/*
ECMAScript:
- official specification / standard of the language

JavaScript:
- real-world implementation based on ECMAScript
*/

// JavaScript engines
/*
Engines execute JavaScript code:
- V8 -> Chrome, Node.js
- SpiderMonkey -> Firefox
- JavaScriptCore -> Safari
*/

// Environment vs Engine
/*
Engine:
- Executes JavaScript

Environment:
- Provides host APIs around the engine

Browser environment gives:
- window
- document
- localStorage
- fetch
- DOM APIs

Node.js environment gives:
- process
- Buffer
- fs
- path
- http
- streams
*/

// Single-threaded nature
/*
JavaScript executes regular synchronous code on one main call stack.

Then how does async work?
- The environment provides APIs (timers, network, DOM events)
- Event loop schedules callbacks later
- So JavaScript feels concurrent even though normal execution is single-threaded
*/

// Interpreted / JIT compiled
/*
Historically JavaScript was called interpreted.
Modern engines usually:
- parse the code
- interpret it
- optimize hot code
- JIT-compile parts for performance

```
Interview-friendly answer:
"Modern JavaScript engines combine parsing, interpretation, and JIT compilation."
*/

// Strict mode
"use strict";
/*
Strict mode:
- catches silent errors
- prevents accidental globals
- changes some unsafe behaviors
*/

// Case sensitivity
const languageName = "JavaScript";
const LanguageName = "Different variable";

// console.log(languageName); // JavaScript
// console.log(LanguageName); // Different variable

// Statements vs Expressions
let a = 10;          // statement containing expression
let sum = 2 + 3;     // expression produces value 5

// Comments
// Single-line comment
/* Multi-line comment */

// Semicolons and ASI
/*
ASI = Automatic Semicolon Insertion

JavaScript can insert semicolons automatically,
but relying on it too much can cause bugs.
*/

function returnObjectBad() {
  return
  {
    ok: false
  };
}

// console.log(returnObjectBad());
// Output: undefined
// Why? ASI inserts semicolon after `return`

function returnObjectGood() {
  return {
```

```
    ok: true
  };
}

// console.log(returnObjectGood());
// Output: { ok: true }


//
================================================================================
==========
// [02] VARIABLES AND DECLARATIONS
//
================================================================================
==========
/*
Main declaration keywords:
- var
- let
- const
*/

// var
/*
var:
- function-scoped
- can be redeclared
- can be reassigned
- hoisted and initialized with undefined

Problems:
- scope leakage
- accidental redeclarations
- confusing hoisting behavior

Modern best practice:
- avoid var in new code unless discussing legacy code
*/
var oldStyle = "old";
var oldStyle = "redeclared"; // allowed

// let
/*
let:
- block-scoped
- can be reassigned
- cannot be redeclared in same scope
- hoisted but uninitialized (TDZ applies)
*/
let age = 25;
```

```
age = 26;

// const
/*
const:
- block-scoped
- cannot be reassigned
- must be initialized immediately

Important:
const does NOT make nested data immutable.
It only prevents rebinding of the variable.
*/
const PI = 3.14159;
// PI = 3.14; // TypeError

const userConst = { name: "Dinesh" };
userConst.name = "Kumar"; // allowed
// console.log(userConst.name); // Kumar

// Variable naming rules
/*
Valid:
- letters
- _ underscore
- $ dollar
- digits (not as first character)

Invalid:
- starts with digit
- reserved keywords
*/
let userName = "valid";
let _private = "valid";
let $price = 100;

// let 1name = "invalid"; // SyntaxError
// let class = "invalid"; // SyntaxError

// Scope
/*
Types:
- Global scope
- Function scope
- Block scope
- Module scope
*/
var globalVar = "global";

function scopeDemo() {
```

```
  var functionScoped = "inside function";

  if (true) {
    let blockScoped = "inside block";
    const alsoBlock = "inside block";
    var stillFunctionScoped = "var ignores block";
  }

  // console.log(functionScoped); // works
  // console.log(stillFunctionScoped); // works
  // console.log(blockScoped); // ReferenceError
}
scopeDemo();

// Hoisting
/*
Hoisting means declarations are processed before execution.

var:
- hoisted
- initialized to undefined

let / const:
- hoisted
- not initialized
- accessing before declaration causes ReferenceError
*/

// console.log(hoistedVar); // undefined
var hoistedVar = "hello";

// console.log(hoistedLet); // ReferenceError
let hoistedLet = "world";

// TDZ
/*
TDZ = Temporal Dead Zone
The period from entering scope until the declaration line is executed
*/
{
  // console.log(tdzExample); // ReferenceError
  let tdzExample = "now initialized";
}

/*
Best practice:
- use const by default
- use let only when reassignment is necessary
- avoid var
*/
```

```
//
================================================================================
// [03] DATA TYPES
//
================================================================================
/*
Primitive types:
1) string
2) number
3) bigint
4) boolean
5) undefined
6) symbol
7) null

Non-primitive:
- object
*/

let str = "hello";            // string
let num = 42;                 // number
let big = 9007199254740993n;  // bigint
let bool = true;              // boolean
let undef;                    // undefined
let sym = Symbol("id");       // symbol
let empty = null;             // null
let obj = { name: "Dinesh" }; // object

// typeof
// console.log(typeof str);   // string
// console.log(typeof num);   // number
// console.log(typeof big);   // bigint
// console.log(typeof bool);  // boolean
// console.log(typeof undef); // undefined
// console.log(typeof sym);   // symbol
// console.log(typeof empty); // object   (historical quirk)
// console.log(typeof obj);   // object
// console.log(typeof function(){}); // function

/*
Important interview question:
typeof null === "object"
This is a historical bug/quirk in JavaScript.
*/

// Primitive vs Reference
```

```
/*
Primitive copy -> by value
Object copy -> by reference
*/
let p1 = 10;
let p2 = p1;
p2 = 20;
// console.log(p1, p2); // 10 20

let user1 = { score: 10 };
let user2 = user1;
user2.score = 99;
// console.log(user1.score, user2.score); // 99 99

// Wrapper objects
/*
Wrapper constructors:
- String
- Number
- Boolean

Avoid new String(), new Number(), new Boolean() in normal code
because they create objects instead of primitive values.
*/
let primitiveString = "abc";
let objectString = new String("abc");

// console.log(typeof primitiveString); // string
// console.log(typeof objectString);    // object
// console.log(primitiveString === objectString); // false


//
==========
// [04] TYPE CONVERSION AND COERCION
//
==========
/*
Why coercion exists:
- to make JavaScript flexible and ergonomic

Problem:
- confusing results in some cases

Best practice:
- use explicit conversion when possible
- use strict equality (===)
*/
```

```
// Explicit conversion
// console.log(String(123)); // "123"
// console.log(Number("123")); // 123
// console.log(Boolean("hello")); // true
// console.log(parseInt("101px", 10)); // 101
// console.log(parseFloat("3.14kg")); // 3.14

// Truthy / Falsy
/*
Falsy values:
- false
- 0
- -0
- 0n
- ""
- null
- undefined
- NaN

Everything else is usually truthy.
*/
if ("hello") {
  // console.log("truthy");
}

// Implicit coercion
// console.log("5" + 1); // "51"
// console.log("5" - 1); // 4
// console.log(true + 1); // 2
// console.log(false + 1); // 1

// Equality
// console.log(5 == "5"); // true
// console.log(5 === "5"); // false
// console.log(null == undefined); // true
// console.log(null === undefined); // false

/*
Best practice:
Prefer === and !==
*/

// NaN
/*
NaN = Not-a-Number
Important:
NaN !== NaN
*/
// console.log(Number("hello")); // NaN
```

```
// console.log(NaN === NaN); // false
// console.log(Number.isNaN(NaN)); // true

// Infinity
// console.log(1 / 0); // Infinity
// console.log(-1 / 0); // -Infinity

// Object.is()
// console.log(Object.is(NaN, NaN)); // true
// console.log(Object.is(+0, -0)); // false
// console.log(+0 === -0); // true


//
==========
// [05] OPERATORS
//
==========

// Arithmetic
let x = 10;
let y = 3;
// console.log(x + y); // 13
// console.log(x - y); // 7
// console.log(x * y); // 30
// console.log(x / y); // 3.333...
// console.log(x % y); // 1
// console.log(x ** y); // 1000

let counter = 1;
counter++;
++counter;
// console.log(counter); // 3

// Assignment
let total = 10;
total += 5;
total -= 2;
total *= 2;
total /= 2;
// console.log(total); // 13

// Comparison
// console.log(5 > 3); // true
// console.log(5 < 3); // false
// console.log(5 >= 5); // true
// console.log(5 <= 4); // false
// console.log(5 == "5"); // true
```

```
// console.log(5 === "5"); // false
// console.log(5 != "5"); // false
// console.log(5 !== "5"); // true

// Logical
// console.log(true && false); // false
// console.log(true || false); // true
// console.log(!true); // false

// Short-circuiting
// console.log(false && "won't run"); // false
// console.log(true || "won't run"); // true

// Ternary
let ageLimit = 20;
let canVote = ageLimit >= 18 ? "Yes" : "No";
// console.log(canVote); // Yes

// typeof
// console.log(typeof 123); // number

// delete
let book = { title: "JS", price: 500 };
delete book.price;
// console.log(book); // { title: 'JS' }

// in
// console.log("title" in book); // true

// instanceof
let arrSample = [];
// console.log(arrSample instanceof Array); // true

// Nullish coalescing ??
let quantity = 0;
// console.log(quantity || 10); // 10 (incorrect if 0 is valid)
// console.log(quantity ?? 10); // 0  (correct)

// Optional chaining ?.
let companyObj = { department: { name: "Engineering" } };
// console.log(companyObj.department?.name); // Engineering
// console.log(companyObj.hr?.name); // undefined

// Spread ...
let arr1 = [1, 2];
let arr2 = [...arr1, 3, 4];
// console.log(arr2); // [1,2,3,4]

let obj1 = { a: 1, b: 2 };
let obj2 = { ...obj1, c: 3 };
```

```
// console.log(obj2); // { a:1, b:2, c:3 }

// Rest ...
function addAll(...nums) {
  return nums.reduce((sum, n) => sum + n, 0);
}
// console.log(addAll(1, 2, 3, 4)); // 10

// Bitwise
// console.log(5 & 1); // 1
// console.log(5 | 1); // 5
// console.log(5 ^ 1); // 4
// console.log(~5);    // -6

// Comma operator
let commaResult = (1, 2, 3);
// console.log(commaResult); // 3


//
==========
// [06] CONTROL FLOW
//
==========
let marks = 78;

if (marks >= 90) {
  // console.log("Grade A");
} else if (marks >= 75) {
  // console.log("Grade B");
} else {
  // console.log("Need improvement");
}

let fruit = "apple";
switch (fruit) {
  case "mango":
    // console.log("Summer fruit");
    break;
  case "apple":
    // console.log("Common fruit");
    break;
  default:
    // console.log("Unknown fruit");
}

function divideSafe(a, b) {
  if (b === 0) return "Cannot divide by zero";
```

```
  return a / b;
}
// console.log(divideSafe(10,2)); // 5
// console.log(divideSafe(10,0)); // Cannot divide by zero

/*
Best practice:
- use early returns
- avoid very deep nesting
- make conditions readable
*/


//
==========
// [07] LOOPS AND ITERATION
//
==========
for (let i = 0; i < 3; i++) {
  // console.log("for", i);
}

let w = 0;
while (w < 2) {
  // console.log("while", w);
  w++;
}

do {
  // console.log("do...while runs once");
} while (false);

// for...in -> keys
let employee = { id: 1, name: "Sam" };
for (let key in employee) {
  // console.log(key, employee[key]);
}

// for...of -> values
for (let value of [10, 20, 30]) {
  // console.log(value);
}

/*
Difference:
- for...in -> keys / property names
- for...of -> iterable values
*/
```

```
outerLoop:
for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (i === 1 && j === 1) break outerLoop;
  }
}
/*
Labels are rare in production because they reduce readability.
*/


//
==========
// [08] STRINGS
//
==========
/*
Strings are immutable.
JS strings are based on UTF-16 code units.
*/
let s1 = "Hello";
let s2 = 'World';
let s3 = `Template literal: ${s1} ${s2}`;
// console.log(s3); // Template literal: Hello World

let multiline = `Line 1
Line 2
Line 3`;
// console.log(multiline);

let original = "cat";
let changed = original.toUpperCase();
// console.log(original); // cat
// console.log(changed); // CAT

let text = "  JavaScript Interview  ";
// console.log(text.length);
// console.log(text.trim()); // "JavaScript Interview"
// console.log(text.toUpperCase());
// console.log(text.toLowerCase());
// console.log(text.slice(2, 12));
// console.log(text.substring(2, 12));
// console.log(text.replace("Interview", "Guide"));
// console.log("a-b-c".replaceAll("-", ":")); // a:b:c
// console.log("a,b,c".split(",")); // ['a','b','c']
// console.log(text.includes("Script")); // true
// console.log(text.startsWith("  J")); // true
```

```
// console.log(text.endsWith("  ")); // true
// console.log(text.indexOf("Script"));
// console.log("5".padStart(3, "0")); // 005
// console.log("5".padEnd(3, "0")); // 500
// console.log("ha".repeat(3)); // hahaha

// Unicode complexity
let smile = "😊";
// console.log(smile.length); // 2 (UTF-16 code units)
// console.log([...smile].length); // 1

// Escape sequences
let escaped = "Hello\nWorld\t!!!";
// console.log(escaped);


//
==========
// [09] NUMBERS AND MATH
//
==========
/*
JavaScript number = IEEE-754 double precision floating-point
*/
// console.log(0.1 + 0.2); // 0.30000000000000004
/*
Use integer minor units or decimal libraries in financial apps.
*/

// console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991
// console.log(Number.MAX_SAFE_INTEGER + 1 === Number.MAX_SAFE_INTEGER +
2); // true

let bigIntValue = 9007199254740993n;
// console.log(bigIntValue + 2n); // 9007199254740995n
// console.log(1n + 1); // TypeError

// Number methods
// console.log(Number.isNaN(NaN)); // true
// console.log(Number.isInteger(10.0)); // true
// console.log(Number.parseInt("42px", 10)); // 42
// console.log(Number.parseFloat("3.14kg")); // 3.14
// console.log((3.14159).toFixed(2)); // "3.14"
// console.log((3.14159).toPrecision(3)); // "3.14"

// Math
// console.log(Math.round(4.6)); // 5
// console.log(Math.floor(4.9)); // 4
```

```
// console.log(Math.ceil(4.1)); // 5
// console.log(Math.trunc(4.9)); // 4
// console.log(Math.max(1,5,2)); // 5
// console.log(Math.min(1,5,2)); // 1
// console.log(Math.pow(2,3)); // 8
// console.log(Math.sqrt(16)); // 4
// console.log(Math.abs(-10)); // 10

function getRandomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
// console.log(getRandomInt(1, 6));


//
================================================================================
==========
// [10] BOOLEANS
//
================================================================================
==========
let isLoggedIn = true;
let hasProfile = false;
// console.log(isLoggedIn && hasProfile); // false
// console.log(isLoggedIn || hasProfile); // true
// console.log(!isLoggedIn); // false

let username = "";
let displayName = username || "Guest";
// console.log(displayName); // Guest


//
================================================================================
==========
// [11] ARRAYS
//
================================================================================
==========
/*
Arrays store ordered collections.
*/
let numbers = [10, 20, 30];
// console.log(numbers[0]); // 10
numbers[1] = 25;
// console.log(numbers); // [10,25,30]
// console.log(numbers.length); // 3

// Sparse arrays
let sparse = [];
```

```
sparse[5] = "X";
// console.log(sparse.length); // 6
// console.log(sparse);

// Multidimensional
let matrix = [
  [1, 2],
  [3, 4]
];
// console.log(matrix[1][0]); // 3

// Mutating methods
let fruits = ["apple", "banana"];
fruits.push("mango");
fruits.pop();
fruits.shift();
fruits.unshift("grape");
fruits.splice(1, 0, "orange");
fruits.reverse();
fruits.sort();
// console.log(fruits);

// sort pitfall
let numsToSort = [100, 2, 20];
numsToSort.sort();
// console.log(numsToSort); // lexical sort incorrect for numeric intent
numsToSort.sort((a, b) => a - b);
// console.log(numsToSort); // [2,20,100]

// Non-mutating methods
let a1 = [1, 2, 3];
let a2 = [4, 5];
// console.log(a1.concat(a2)); // [1,2,3,4,5]
// console.log(a1.slice(1)); // [2,3]
// console.log(a1.map(x => x * 2)); // [2,4,6]
// console.log(a1.filter(x => x > 1)); // [2,3]
// console.log(a1.reduce((sum, x) => sum + x, 0)); // 6
// console.log(a1.find(x => x > 1)); // 2
// console.log(a1.findIndex(x => x === 3)); // 2
// console.log(a1.some(x => x > 2)); // true
// console.log(a1.every(x => x > 0)); // true
// console.log(a1.includes(2)); // true
// console.log(a1.join("-")); // "1-2-3"
// console.log([1, [2, [3]]].flat(2)); // [1,2,3]
// console.log([1,2].flatMap(x => [x, x * 10])); // [1,10,2,20]

// Modern non-mutating array methods
let arrModern = [3, 1, 2];
// console.log(arrModern.toSorted((a, b) => a - b)); // [1,2,3]
// console.log(arrModern.toReversed()); // [2,1,3]
```

```
// console.log(arrModern.toSpliced(1, 1, 99)); // [3,99,2]
// console.log(arrModern); // original unchanged

// Iteration methods
[1, 2, 3].forEach((value, index) => {
  // console.log(index, value);
});

// Destructuring
let [first, second, ...restArr] = [10, 20, 30, 40];
// console.log(first, second, restArr); // 10 20 [30,40]

// Shallow copy
let nested1 = [{ val: 1 }];
let nested2 = [...nested1];
nested2[0].val = 999;
// console.log(nested1[0].val); // 999

// Deep copy
let originalNested = { user: { name: "A" } };
let deepCopy = structuredClone(originalNested);
deepCopy.user.name = "B";
// console.log(originalNested.user.name); // A
// console.log(deepCopy.user.name); // B


//
==========
// [12] OBJECTS
//
==========
/*
Objects store key-value pairs.
*/
let person = {
  name: "Dinesh",
  age: 30,
  greet() {
    return `Hello, ${this.name}`;
  }
};
// console.log(person.name); // Dinesh
// console.log(person["age"]); // 30
// console.log(person.greet()); // Hello, Dinesh

// Dynamic property
let keyName = "city";
person[keyName] = "Coimbatore";
```

```
// console.log(person.city); // Coimbatore

// Computed property names
let dynamicKey = "score";
let resultObj = {
  [dynamicKey]: 95
};
// console.log(resultObj.score); // 95

// Nested objects
let companyInfo = {
  name: "ABC",
  address: {
    city: "Chennai",
    pin: 600001
  }
};
// console.log(companyInfo.address.city); // Chennai

// Symbol keys
let secretKey = Symbol("secret");
let secured = {
  [secretKey]: "hiddenValue"
};
// console.log(secured[secretKey]); // hiddenValue

// Utilities
// console.log(Object.keys(person));
// console.log(Object.values(person));
// console.log(Object.entries(person));

let merged = Object.assign({}, { a: 1 }, { b: 2 });
// console.log(merged); // { a:1, b:2 }

// freeze / seal / preventExtensions
let settings = { theme: "dark" };
Object.freeze(settings);
// settings.theme = "light";
// console.log(settings.theme); // dark

let profile = { name: "Sam" };
Object.seal(profile);
profile.name = "John";
// delete profile.name; // not allowed

let config = { mode: "dev" };
Object.preventExtensions(config);
// config.newProp = 123; // not allowed

// hasOwnProperty / Object.hasOwn
```

```
// console.log(person.hasOwnProperty("name")); // true
// console.log(Object.hasOwn(person, "name")); // true

// Property descriptors
let descriptorObj = {};
Object.defineProperty(descriptorObj, "id", {
  value: 101,
  writable: false,
  enumerable: true,
  configurable: false
});
// descriptorObj.id = 999;
// console.log(descriptorObj.id); // 101

// Getters / setters
let temperature = {
  _celsius: 0,
  get fahrenheit() {
    return this._celsius * 9 / 5 + 32;
  },
  set celsius(value) {
    if (typeof value !== "number") throw new TypeError("Must be number");
    this._celsius = value;
  }
};
temperature.celsius = 25;
// console.log(temperature.fahrenheit); // 77

// Object.create
let animalProto = {
  speak() {
    return `${this.name} makes a sound`;
  }
};
let dogObj = Object.create(animalProto);
dogObj.name = "Tommy";
// console.log(dogObj.speak()); // Tommy makes a sound


//
================================================================
==========
// [13] FUNCTIONS
//
================================================================
==========
/*
Functions are first-class:
- stored in variables
- passed as arguments
```

```
- returned from functions
*/

// Function declaration
function add(x, y) {
  return x + y;
}
// console.log(add(2, 3)); // 5

// Function expression
const subtract = function (x, y) {
  return x - y;
};
// console.log(subtract(5, 2)); // 3

// Named function expression
const factorialExpr = function factorial(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
};
// console.log(factorialExpr(5)); // 120

// Arrow function
const multiply = (x, y) => x * y;
// console.log(multiply(3, 4)); // 12
/*
Arrow functions:
- concise
- lexical this
- no own arguments
- cannot be constructors
*/

// Default parameters
function greet(name = "Guest") {
  return `Hello, ${name}`;
}
// console.log(greet()); // Hello, Guest

// Rest parameters
function sumAll(...values) {
  return values.reduce((sum, val) => sum + val, 0);
}
// console.log(sumAll(1,2,3)); // 6

// Higher-order function
function operate(a, b, fn) {
  return fn(a, b);
}
// console.log(operate(10, 5, add)); // 15
```

```
// Callback
setTimeout(() => {
  // console.log("I am a callback");
}, 10);

// Pure function
function pureAdd(a, b) {
  return a + b;
}

// Side effect
let externalCounter = 0;
function incrementCounter() {
  externalCounter++;
}

// Recursion
function factorial(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
}
// console.log(factorial(5)); // 120

// IIFE
(function () {
  let hidden = "private scope";
  // console.log(hidden);
})();

// arguments object
function showArguments() {
  // console.log(arguments);
}
showArguments(1, 2, 3);

// Function hoisting
hoistedFunction();
function hoistedFunction() {
  // console.log("Function declarations are hoisted");
}

// Closure
function outer() {
  let count = 0;
  return function inner() {
    count++;
    return count;
  };
}
```

```
const counterFn = outer();
// console.log(counterFn()); // 1
// console.log(counterFn()); // 2
/*
Closure = function remembers lexical variables even after outer function returns
Use cases:
- private state
- factory functions
- memoization
- event handlers
*/

// Scope chain
let globalMessage = "global";
function one() {
  let local1 = "one";
  function two() {
    let local2 = "two";
    // console.log(globalMessage, local1, local2);
  }
  two();
}
one();

// call / apply / bind
function introduce(city, country) {
  return `I am ${this.name} from ${city}, ${country}`;
}
const dev = { name: "Dinesh" };
// console.log(introduce.call(dev, "Coimbatore", "India"));
// console.log(introduce.apply(dev, ["Coimbatore", "India"]));
const boundIntroduce = introduce.bind(dev, "Coimbatore");
// console.log(boundIntroduce("India"));

// Currying
function curryAdd(a) {
  return function (b) {
    return a + b;
  };
}
// console.log(curryAdd(2)(3)); // 5

// Partial application
function volume(l, w, h) {
  return l * w * h;
}
const areaLike = volume.bind(null, 10, 5);
// console.log(areaLike(2)); // 100

// Function composition
```

```
const double = x => x * 2;
const square = x => x * x;
const compose = (f, g) => x => f(g(x));
const squareThenDouble = compose(double, square);
// console.log(squareThenDouble(3)); // 18

// Memoization
function createMemoizedSquare() {
  const cache = new Map();
  return function (n) {
    if (cache.has(n)) return cache.get(n);
    const result = n * n;
    cache.set(n, result);
    return result;
  };
}
const memoSquare = createMemoizedSquare();
// console.log(memoSquare(5)); // 25
// console.log(memoSquare(5)); // cached 25

// Debounce
function debounce(fn, delay) {
  let timerId;
  return function (...args) {
    clearTimeout(timerId);
    timerId = setTimeout(() => fn.apply(this, args), delay);
  };
}
/*
Use case:
search input -> wait until typing stops
*/

// Throttle
function throttle(fn, interval) {
  let lastTime = 0;
  return function (...args) {
    const now = Date.now();
    if (now - lastTime >= interval) {
      lastTime = now;
      fn.apply(this, args);
    }
  };
}
/*
Use case:
scroll / resize handlers
*/
```

```
//
================================================================================
// [14] SCOPE AND EXECUTION CONTEXT
//
================================================================================
/*
Execution context:
Internal engine structure for running code

Types:
- Global execution context
- Function execution context
- Eval execution context (rare, avoid eval)
*/

// Call stack
function firstFn() {
  secondFn();
}
function secondFn() {
  thirdFn();
}
function thirdFn() {
  // console.log("Reached thirdFn");
}
// firstFn();

/*
Call stack is LIFO (Last In, First Out)
*/

// Stack overflow example
function recurseForever() {
  return recurseForever();
}
// recurseForever(); // RangeError


//
================================================================================
// [15] THIS KEYWORD
//
================================================================================
/*
`this` depends on how a function is called.
*/
```

```
function regularFunctionThis() {
  return this;
}
// console.log(regularFunctionThis()); // undefined in strict mode

const userThis = {
  name: "Alex",
  show() {
    return this.name;
  }
};
// console.log(userThis.show()); // Alex

function Person(name) {
  this.name = name;
}
const personInstance = new Person("Sam");
// console.log(personInstance.name); // Sam

const arrowThisDemo = {
  name: "Outer",
  regular() {
    const arrow = () => this.name;
    return arrow();
  }
};
// console.log(arrowThisDemo.regular()); // Outer

/*
Rules summary:
1) Plain function in strict mode -> undefined
2) Method call obj.fn() -> obj
3) Constructor with new -> new instance
4) call/apply/bind -> explicitly chosen this
5) Arrow function -> lexical this
*/


//
==========
// [16] PROTOTYPES AND INHERITANCE
//
==========
/*
JavaScript uses prototype-based inheritance.
*/
function Animal(name) {
  this.name = name;
```

```
}
Animal.prototype.speak = function () {
  return `${this.name} makes a sound`;
};

const dog = new Animal("Dog");
// console.log(dog.speak()); // Dog makes a sound
// console.log(dog.hasOwnProperty("name")); // true
// console.log(dog.hasOwnProperty("speak")); // false

/*
prototype property:
- exists on constructor functions
- instances created with new link to constructor.prototype

__proto__:
- historical accessor
- prefer Object.getPrototypeOf / Object.setPrototypeOf
*/

// Inheritance
function Dog(name, breed) {
  Animal.call(this, name);
  this.breed = breed;
}
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;

Dog.prototype.bark = function () {
  return `${this.name} barks`;
};

const lab = new Dog("Buddy", "Labrador");
// console.log(lab.speak()); // Buddy makes a sound
// console.log(lab.bark()); // Buddy barks

// Property shadowing
Animal.prototype.type = "generic animal";
lab.type = "domestic dog";
// console.log(lab.type); // domestic dog


//
==========
// [17] CLASSES
//
==========
/*
```

```
Classes are syntactic sugar over prototypes.
*/
class Vehicle {
  static category = "Transport";
  #secretCode = 12345;

  constructor(brand) {
    this.brand = brand;
  }

  start() {
    return `${this.brand} started`;
  }

  get secret() {
    return this.#secretCode;
  }

  set secret(value) {
    if (typeof value !== "number") throw new TypeError("Secret must be number");
    this.#secretCode = value;
  }

  static info() {
    return "Vehicles help transportation";
  }
}

class Car extends Vehicle {
  constructor(brand, model) {
    super(brand);
    this.model = model;
  }

  start() {
    return `${super.start()} -> model ${this.model}`;
  }
}

const myCar = new Car("Toyota", "Corolla");
// console.log(myCar.start()); // Toyota started -> model Corolla
// console.log(Car.info()); // Vehicles help transportation
// console.log(Vehicle.category); // Transport
// console.log(myCar.secret); // 12345

// Class expression
const Box = class {
  constructor(value) {
    this.value = value;
  }
```

```
};
const bx = new Box(10);
// console.log(bx.value); // 10


//
============================================================================
==========
// [18] ERROR HANDLING
//
============================================================================
==========
/*
Error types:
- SyntaxError
- ReferenceError
- TypeError
- RangeError
- URIError
- EvalError
- generic Error
*/
try {
  JSON.parse("invalid json");
} catch (error) {
  // console.log(error.name); // SyntaxError
  // console.log(error.message);
} finally {
  // console.log("cleanup if needed");
}

function divideStrict(a, b) {
  if (b === 0) throw new RangeError("Denominator cannot be zero");
  return a / b;
}
// console.log(divideStrict(10,2)); // 5

class ValidationError extends Error {
  constructor(message, field) {
    super(message);
    this.name = "ValidationError";
    this.field = field;
  }
}
function validateEmail(email) {
  if (!email.includes("@")) {
    throw new ValidationError("Invalid email format", "email");
  }
  return true;
}
```

```
try {
  validateEmail("abc.com");
} catch (err) {
  // console.log(err.name, err.field, err.message);
}

/*
Best practice:
- throw meaningful errors
- catch only where you can handle them
- do not silently swallow errors
*/


//
==========
// [19] DATES AND TIME
//
==========
/*
Date stores milliseconds since Unix epoch (Jan 1 1970 UTC)
*/
let now = new Date();
// console.log(now.toString());
// console.log(now.toISOString());

let timestamp = Date.now();
// console.log(timestamp);

let parsedDate = new Date("2026-06-17T10:00:00Z");
// console.log(parsedDate.getUTCFullYear()); // 2026
// console.log(parsedDate.getMonth()); // 0-based month

let future = new Date(now);
future.setDate(now.getDate() + 7);
// console.log(future.toDateString());

/*
Date limitations:
- timezone complexity
- parsing ambiguity
- DST issues
Best practice:
- store UTC/timestamps
- format only for display
*/
```

```
//
================================================================================
// [20] REGULAR EXPRESSIONS
//
================================================================================
/*
Regex = pattern matching for strings
*/
let regex = /cat/;
// console.log(regex.test("my cat")); // true

let regexCaseInsensitive = /hello/i;
// console.log(regexCaseInsensitive.test("HELLO")); // true

// Character classes
// console.log(/[0-9]+/.test("123")); // true

// Quantifiers
// console.log(/a{2,4}/.test("aaa")); // true

// Groups and alternation
let petRegex = /(cat|dog)/;
// console.log(petRegex.exec("I have a dog"));

// Anchors
// console.log(/^hello$/.test("hello")); // true exact match

// Lookahead / Lookbehind
// console.log(/\d+(?=px)/.exec("20px")); // matches 20
// console.log(/(?<=\$)\d+/.exec("$100")); // matches 100 if environment supports it

// String methods with regex
// console.log("abc123".match(/\d+/));
// console.log("abc123".replace(/\d+/, "XYZ")); // abcXYZ
// console.log("a,b;c".split(/[;,]/)); // ['a','b','c']


//
================================================================================
// [21] JSON
//
================================================================================
/*
JSON = JavaScript Object Notation
Text format for data exchange
*/
```

```
let data = { name: "Dinesh", age: 30, skills: ["JS", "React"] };
let jsonString = JSON.stringify(data);
// console.log(jsonString);
let parsed = JSON.parse(jsonString);
// console.log(parsed.name); // Dinesh

/*
JSON limitations:
- no functions
- no undefined
- no Symbol
- no circular references
- Date becomes string
*/
let weird = {
  a: undefined,
  b: function () {},
  c: Symbol("x")
};
// console.log(JSON.stringify(weird));


//
==========
// [22] ADVANCED BUILT-IN OBJECTS
//
==========

// Map
const map = new Map();
map.set("name", "Dinesh");
map.set({ id: 1 }, "object key value");
// console.log(map.get("name")); // Dinesh
// console.log(map.size); // 2

// Set
const set = new Set([1, 2, 2, 3]);
// console.log(set); // Set {1,2,3}
set.add(4);
set.delete(2);
// console.log(set.has(3)); // true

// WeakMap
const wm = new WeakMap();
const objKey = {};
wm.set(objKey, "private data");

// Symbol
```

```
const uniqueId = Symbol("id");
const userWithSymbol = {
  [uniqueId]: 123
};
// console.log(userWithSymbol[uniqueId]); // 123

// Iterable protocol / iterator protocol
const myIterable = {
  data: [10, 20, 30],
  [Symbol.iterator]() {
    let index = 0;
    let arr = this.data;
    return {
      next() {
        if (index < arr.length) {
          return { value: arr[index++], done: false };
        }
        return { value: undefined, done: true };
      }
    };
  }
};
for (const value of myIterable) {
  // console.log(value);
}

// Generator
function* idGenerator() {
  yield 1;
  yield 2;
  yield 3;
}
const gen = idGenerator();
// console.log(gen.next()); // { value:1, done:false }
// console.log(gen.next()); // { value:2, done:false }

// Async generator
async function* asyncCounter() {
  yield 1;
  yield 2;
}
// for await (const value of asyncCounter()) { console.log(value); }

// ArrayBuffer / TypedArray
const buffer = new ArrayBuffer(8);
const view = new Uint8Array(buffer);
view[0] = 255;
// console.log(view[0]); // 255

// Proxy + Reflect
```

```
const target = { count: 0 };
const proxy = new Proxy(target, {
  get(obj, prop, receiver) {
    return Reflect.get(obj, prop, receiver);
  },
  set(obj, prop, value, receiver) {
    if (prop === "count" && value < 0) {
      throw new RangeError("count cannot be negative");
    }
    return Reflect.set(obj, prop, value, receiver);
  }
});
proxy.count = 5;
// console.log(proxy.count); // 5

// Intl
const currencyFormatter = new Intl.NumberFormat("en-IN", {
  style: "currency",
  currency: "INR"
});
// console.log(currencyFormatter.format(123456.78)); // ₹1,23,456.78


//
==========
// [23] DESTRUCTURING
//
==========
const coords = [12.34, 56.78];
const [lat, lng] = coords;
// console.log(lat, lng);

const student = { name: "Raj", marks: 95, city: "Madurai" };
const { name: studentName, marks: marks2 = 0, city = "Unknown" } = student;
// console.log(studentName, marks2, city);

const nestedObj = { user: { profile: { title: "Developer" } } };
const {
  user: {
    profile: { title }
  }
} = nestedObj;
// console.log(title); // Developer

const [head, ...tail] = [1, 2, 3, 4];
// console.log(head, tail); // 1 [2,3,4]
```

```
//
================================================================================
==========
// [24] SPREAD AND REST
//
================================================================================
==========
const numsA = [1, 2];
const numsB = [...numsA, 3, 4];
// console.log(numsB); // [1,2,3,4]

const mergedObj = { ...{ a: 1 }, ...{ b: 2 } };
// console.log(mergedObj); // {a:1,b:2}

function maxOfValues(...vals) {
  return Math.max(...vals);
}
// console.log(maxOfValues(1, 9, 3)); // 9

const { a: firstProp, ...remainingProps } = { a: 1, b: 2, c: 3 };
// console.log(firstProp, remainingProps); // 1 {b:2,c:3}


/*
Spread:
- expands items

Rest:
- collects remaining items
*/


//
================================================================================
==========
// [25] MODULES
//
================================================================================
==========
/*
Why modules?
- prevent global namespace pollution
- improve organization
- improve reuse
- support lazy loading and tree shaking
*/

/*
ES Modules example:
```

```
// math.js
export const PI2 = 3.14159;
export function add2(a, b) { return a + b; }
export default function multiply2(a, b) { return a * b; }

// app.js
import multiply2, { PI2, add2 as sum2 } from "./math.js";
console.log(PI2, sum2(2,3), multiply2(2,3));
*/

/*
CommonJS example:

// math.cjs
module.exports = { add: (a,b) => a+b };

// app.cjs
const { add } = require("./math.cjs");
console.log(add(2,3));
*/

/*
ESM vs CommonJS:
- ESM is static and standard
- CommonJS is older Node system
- ESM enables tree shaking more naturally
*/

/*
Circular dependency warning:
If A imports B and B imports A,
partially initialized values can appear.
Best fix:
- refactor shared logic into third module
*/


//
==========
// [26] ASYNCHRONOUS JAVASCRIPT
//
==========
/*
Synchronous code:
- line-by-line blocking execution

Asynchronous code:
- starts operation and continues without waiting
```

```
*/
setTimeout(() => {
  // console.log("setTimeout after delay");
}, 10);

const intervalId = setInterval(() => {
  // console.log("interval tick");
  clearInterval(intervalId);
}, 10);

// Promise basics
const promiseDemo = new Promise((resolve, reject) => {
  let success = true;
  if (success) resolve("Operation successful");
  else reject(new Error("Operation failed"));
});

promiseDemo
  .then(result => {
    // console.log(result);
  })
  .catch(error => {
    // console.error(error.message);
  })
  .finally(() => {
    // console.log("Promise completed");
  });

/*
Promise states:
- pending
- fulfilled
- rejected
*/

// Static methods
const p1Demo = Promise.resolve(1);
const p2Demo = Promise.resolve(2);
const p3Demo = Promise.reject(new Error("fail"));

Promise.all([p1Demo, p2Demo]).then(values => {
  // console.log(values); // [1,2]
});

Promise.allSettled([p1Demo, p3Demo]).then(results => {
  // console.log(results);
});

Promise.race([
  new Promise(res => setTimeout(() => res("fast"), 10)),
```

```
  new Promise(res => setTimeout(() => res("slow"), 50))
]).then(value => {
  // console.log(value); // fast
});

Promise.any([
  Promise.reject("A"),
  Promise.resolve("B")
]).then(value => {
  // console.log(value); // B
});

// async / await
function fakeApi(value, delay) {
  return new Promise(resolve => setTimeout(() => resolve(value), delay));
}

async function asyncExample() {
  try {
    const a = await fakeApi("first", 10);
    const b = await fakeApi("second", 10);
    return [a, b];
  } catch (err) {
    throw err;
  }
}
// asyncExample().then(console.log); // ['first','second']

// parallel async
async function parallelExample() {
  const [a, b] = await Promise.all([
    fakeApi("first", 10),
    fakeApi("second", 10)
  ]);
  return [a, b];
}
// parallelExample().then(console.log);

// Cancellation pattern
const controller = new AbortController();
const signal = controller.signal;
// fetch('/api/data', { signal });
// controller.abort();


//
==========
// [27] EVENT LOOP AND SCHEDULING
```

```
//
==========
/*
Core pieces:
- Call stack
- Host APIs
- Task queue / macrotasks
- Microtask queue
- Event loop

Microtasks:
- Promise callbacks
- queueMicrotask

Macrotasks:
- setTimeout
- setInterval
- IO
- DOM events
*/

// console.log("start");
// setTimeout(() => console.log("timeout"), 0);
// Promise.resolve().then(() => console.log("promise"));
// queueMicrotask(() => console.log("microtask"));
// console.log("end");

/*
Typical output:
start
end
promise
microtask
timeout
*/


//
==========
// [28] DOM AND BROWSER APIS (BROWSER ONLY)
//
==========
/*
DOM = Document Object Model
BOM = Browser Object Model

Selecting:
```

- getElementById
- getElementsByClassName
- getElementsByTagName
- querySelector
- querySelectorAll

Manipulating:
- textContent
- innerText
- innerHTML
- classList
- style
- createElement
- appendChild
- remove
*/

/*
Browser example:

const el = document.getElementById("title");
console.log(el.textContent);

const btn = document.querySelector(".btn");
btn.textContent = "Clicked";
btn.classList.add("active");
btn.style.color = "red";

const div = document.createElement("div");
div.textContent = "Hello DOM";
document.body.appendChild(div);
*/

/*
Important:
- textContent is safe for plain text
- innerHTML parses HTML and can cause XSS if input is untrusted
*/

//
==========
// [29] EVENTS (BROWSER ONLY)
//
==========
/*
Common events:
- click

- dblclick
- input
- change
- submit
- keydown
- keyup
- mouseover
- mouseout
- focus
- blur
- load
- scroll
*/

/*
Event flow:
1) Capturing
2) Target
3) Bubbling
*/

/*
Browser example:

const btn = document.querySelector("#saveBtn");
btn.addEventListener("click", function(event) {
  console.log("Clicked");
  console.log(event.type);
});
*/

/*
preventDefault():
- stops default browser action

stopPropagation():
- stops event from bubbling/capturing further
*/

/*
Event delegation:
Attach one listener on a parent
Handle child events via event.target
Useful for dynamic lists and performance
*/

/*
Example:
const list = document.querySelector("#todoList");
list.addEventListener("click", function(event) {

```
  if (event.target.matches("li")) {
    console.log("Clicked:", event.target.textContent);
  }
});
*/


//
==========
// [30] FORMS AND VALIDATION (BROWSER ONLY)
//
==========
/*
Form handling:
- collect input values
- validate data
- prevent bad submissions

Constraint Validation API:
- required
- minlength
- maxlength
- pattern
- type="email"
- checkValidity()
- reportValidity()
*/

/*
Browser example:

const form = document.querySelector("#signupForm");

form.addEventListener("submit", (e) => {
  e.preventDefault();

  if (!form.checkValidity()) {
    form.reportValidity();
    return;
  }

  const fd = new FormData(form);
  console.log(fd.get("email"));
});
*/
```

```
//
================================================================
==========
// [31] BROWSER STORAGE (BROWSER ONLY)
//
================================================================
==========
/*
localStorage:
- persists until cleared

sessionStorage:
- persists for tab session

cookies:
- small text data
- may be sent with HTTP requests

IndexedDB:
- structured asynchronous client-side database
*/

/*
localStorage example:
localStorage.setItem("theme", "dark");
console.log(localStorage.getItem("theme"));
localStorage.removeItem("theme");
*/

/*
Store object:
const settings2 = { mode: "dark" };
localStorage.setItem("settings", JSON.stringify(settings2));
const saved = JSON.parse(localStorage.getItem("settings"));
console.log(saved.mode);
*/

/*
Limitations:
- localStorage / sessionStorage store strings
- cookies have size overhead
- IndexedDB is powerful but more complex
*/


//
================================================================
==========
// [32] BROWSER NETWORKING APIS
```

```
//
==========
/*
Fetch API:
modern promise-based HTTP API
*/

/*
fetch("/api/users")
  .then(response => {
    if (!response.ok) throw new Error("HTTP error");
    return response.json();
  })
  .then(data => console.log(data))
  .catch(err => console.error(err));
*/

/*
async function loadUsers() {
  const response = await fetch("/api/users");
  if (!response.ok) throw new Error("Request failed");
  return response.json();
}
*/

/*
XMLHttpRequest:
older API still seen in legacy code

AJAX:
general idea of async requests without full page reload
*/

/*
CORS:
Cross-Origin Resource Sharing
Browser security rule for cross-origin requests
*/

/*
WebSockets:
persistent full-duplex communication
Use cases:
- chat
- dashboards
- multiplayer / live features
*/

/*
```

SSE:
Server-Sent Events
One-way stream from server to client
Useful for notifications and feeds
*/

//
==========
// [33] BROWSER ADVANCED APIS (BROWSER ONLY)
//
==========
/*
Geolocation API
Canvas API
Web Workers
Service Workers
Notifications API
Clipboard API
Intersection Observer
Mutation Observer
Resize Observer
History API
Location API
*/

/*
Use cases:
- IntersectionObserver -> lazy loading / infinite scroll
- Web Workers -> heavy computation without UI freeze
- Service Workers -> offline caching / PWA
- ResizeObserver -> responsive UI logic
*/

//
==========
// [34] NODE.JS CONCEPTS (NODE ONLY)
//
==========
/*
Node.js:
JavaScript runtime outside browser
Built on V8 + Node APIs
*/

```
/*
Why use Node?
- backend APIs
- scripts
- tooling
- automation
- real-time systems
*/

// Global objects
/*
- global
- process
- Buffer
- setTimeout
*/

// process
/*
Provides:
- environment variables
- current working directory
- argv
- process control
*/

/*
Node example:
console.log(process.env.NODE_ENV);
console.log(process.cwd());
console.log(process.argv);
*/

// Buffer
/*
Used for raw binary data
*/

// fs
/*
File system module
*/

/*
Example:
const fs = require("fs");
fs.writeFileSync("demo.txt", "Hello Node");
console.log(fs.readFileSync("demo.txt", "utf8"));
*/
```

```
// path
/*
Utilities for working with file paths
*/

// http
/*
Can create HTTP server
*/

/*
Example:
const http = require("http");
const server = http.createServer((req, res) => {
  res.end("Hello");
});
server.listen(3000);
*/

// streams
/*
Process data in chunks
Useful for large files / low memory usage
*/

// EventEmitter
/*
const EventEmitter = require("events");
const emitter = new EventEmitter();

emitter.on("greet", (name) => {
  console.log("Hello", name);
});

emitter.emit("greet", "Dinesh");
*/

// package.json
/*
Stores:
- name/version
- scripts
- dependencies
- metadata
*/

// npm / yarn / pnpm
/*
Package managers
*/
```

```
// child_process / worker_threads
/*
child_process:
- run subprocesses

worker_threads:
- real threads for CPU-heavy work
*/

// CommonJS vs ESM
/*
CommonJS:
- require
- module.exports

ESM:
- import
- export
*/


//
==========
// [35] MEMORY AND PERFORMANCE
//
==========
/*
Stack vs Heap

Stack:
- function call frames
- fast access

Heap:
- objects, dynamic allocations
*/

// Primitive vs reference storage
/*
Primitives are value-like
Objects are referenced
*/

// Garbage collection
/*
JavaScript automatically frees unreachable memory
Common idea: mark-and-sweep
```

```
*/

// Memory leaks
/*
Common causes:
- forgotten intervals/timeouts
- event listeners not removed
- accidental globals
- closures keeping large data alive
- detached DOM nodes
*/

function startInterval() {
  const id = setInterval(() => {
    // repeated work
  }, 1000);

  return () => clearInterval(id);
}
const stopInterval = startInterval();
stopInterval();

// Big-O basics
function getFirst(arr) {
  return arr[0]; // O(1)
}

function findValue(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === target) return i;
  }
  return -1; // O(n)
}

function bubbleSortLike(arr) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = 0; j < arr.length - 1; j++) {
      if (arr[j] > arr[j + 1]) {
        [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];
      }
    }
  }
  return arr; // O(n^2)
}

/*
Lazy loading:
- load when needed

Code splitting:
```

```
- split bundles for faster startup

Memoization:
- cache expensive calculations

Debounce / Throttle:
- control high-frequency calls
*/

/*
Reflow:
- layout recalculation

Repaint:
- visual redraw

Too many reflows hurt UI performance
*/


//
==========
// [36] FUNCTIONAL PROGRAMMING IN JAVASCRIPT
//
==========
/*
Concepts:
- first-class functions
- higher-order functions
- pure functions
- immutability
- referential transparency
- composition
*/

const addTax = price => price * 1.18;
const formatPrice = price => `₹${price.toFixed(2)}`;
const compose2 = (f, g) => x => f(g(x));
const formatFinalPrice = compose2(formatPrice, addTax);
// console.log(formatFinalPrice(100)); // ₹118.00

// Immutability
const state = {
  count: 0,
  user: { name: "A" }
};
const nextState = {
  ...state,
```

```
  count: state.count + 1,
  user: { ...state.user, name: "B" }
};
// console.log(state);
// console.log(nextState);

function pureDouble(x) {
  return x * 2;
}
/*
Referential transparency:
pureDouble(4) can be replaced by 8 without changing program meaning
*/


//
==========
// [37] PRACTICAL INTERVIEW SCENARIOS
//
==========
/*
1) Why prefer let/const over var?
- block scoping
- fewer bugs
- clearer intent

2) Difference between == and ===?
- == coerces
- === compares type + value

3) Why is 0.1 + 0.2 not exactly 0.3?
- floating point precision issue

4) What is closure?
- inner function remembers lexical variables after outer returns

5) Difference between for...in and for...of?
- for...in -> keys
- for...of -> values

6) What is event delegation?
- parent listener handles child events using bubbling

7) What is prototype chain?
- property lookup through linked prototypes

8) Sync vs async?
- sync blocks
```

- async continues and resumes later

9) Promise vs async/await?
- async/await is cleaner syntax built on promises

10) Why use Map over Object?
- keys of any type
- insertion order
- better frequent add/remove semantics
*/

//
==========
// [38] COMMON PITFALLS CHECKLIST
//
==========
/*
- typeof null === "object"
- NaN !== NaN
- [] == false is true
- default sort() is lexical
- return newline object -> ASI issue
- arrow functions don't have their own this
- shallow copy doesn't clone nested objects
- localStorage stores strings
- Promise callbacks run before setTimeout callbacks
- deleting array element leaves a hole
- class syntax still uses prototypes under the hood
*/

//
==========
// [39] MINI PRACTICE QUESTIONS
//
==========
/*
Q1:
console.log(typeof NaN);
A: "number"

Q2:
console.log([] == false);
A: true

Q3:
console.log("5" - 2, "5" + 2);
A: 3 and "52"

Q4:
Explain closure with code.

Q5:
Difference between shallow copy and deep copy?

Q6:
When use Promise.all vs Promise.allSettled?

Q7:
Why avoid innerHTML with untrusted input?
A: XSS risk

Q8:
Why debounce a search box?
A: reduce unnecessary API calls
*/

//
==========
// [40] FINAL RECALL MAP
//
==========
/*
Basics:
- JS is dynamic, single-threaded, prototype-based

Variables:
- prefer const, then let
- understand scope / hoisting / TDZ

Types:
- 7 primitives + object
- know coercion and equality traps

Functions:
- first-class
- closure
- this
- HOFs
- callbacks

Objects:

- properties
- methods
- descriptors
- prototypes
- classes

Async:
- callbacks -> promises -> async/await
- event loop
- microtasks vs macrotasks

Browser:
- DOM
- events
- forms
- storage
- fetch
- browser APIs

Node:
- process
- Buffer
- fs
- streams
- modules

Performance:
- memory
- GC
- leaks
- Big-O
- debounce / throttle

FP:
- pure functions
- immutability
- composition
*/

// End of guide
```
```